Automatización de una aplicación híbrida de riesgo de crédito: un comando, un hook y una extensión

Jaime Lopez
Atlantis University · MAI 540: Machine Learning
Profesor: Kevin A. García Gallardo
Tarea 7.1 · 18 de octubre de 2026

La aplicación híbrida de la Clase 7 predice con un Random Forest la probabilidad de que un crédito sea malo, la explica con un modelo de lenguaje y verifica el texto antes de mostrarlo. Sobre el conjunto German Credit (Hofmann, 1994) se agregaron tres piezas de automatización, cada una con su evidencia en `evidencia/`.

Comando propio: vivienda

`python tareas.py vivienda` reemplaza una tarea que repetía a mano: auditar el predictor por grupo, lo que exige entrenar el modelo, separar la prueba, agrupar y acordarse de mirar el tamaño de cada grupo. Con la semilla 42, la exactitud es .77 en los tres tipos de vivienda, pero el modelo detecta solo .26 de los créditos malos de quienes tienen vivienda propia (*n* = 143), frente a .62 en alquilada (*n* = 35) y .56 en sin costo (*n* = 22, marcado como poco fiable por tener menos de 30 casos). Conviene tenerlo como comando porque la comparación se repite idéntica y el aviso de muestra pequeña no depende de la memoria de quien audita.

Hook de Git

`hooks/pre-commit` ejecuta dos comprobaciones antes de cada commit: `pytest` y una propia que cancela si queda algún bloque TODO sin completar. La segunda existe porque las pruebas solo importan `predictor.py` y `verificador.py`: al dejar un TODO en `red_neuronal.py` pytest siguió en verde (5 pruebas) y únicamente la segunda comprobación canceló el commit. La primera se demostró cambiando 0.30 por 0.99 en `verificar_tono`, lo que hizo fallar una prueba y cancelar el commit.

Extensión auditoria-equidad

Empaqueta el comando `/auditoria-equidad:vivienda`, que ejecuta el anterior y responde con una ficha de cuatro encabezados (hallazgo, evidencia, fiabilidad, siguiente comprobación), y el

agente de solo lectura `auditor-de-equidad`, que contrasta una afirmación sobre un grupo con el código y los datos. Ambos difieren de los de clase, que auditaban por edad y revisaban explicaciones. `verificar_extension.py` no reporta ninguna línea FALTA. Al cargarla, `plugin.json` fija el nombre que prefija los comandos, `vivienda.md` define qué se ejecuta y el agente recibe su rol y sus herramientas de lectura. Con `claude --plugin-dir extension` el comando devolvió la ficha y el agente concluyó que «es justo con todos» no está sustentado.

Riesgo que reduce cada pieza y dónde no protege

Tabla 1

Riesgo que evita cada pieza y un caso en el que no protege

Pieza	Riesgo que reduce	Caso en que no protege
Comando vivienda	Quedarse con la exactitud global (.77 en los tres grupos) y olvidar el tamaño de cada grupo; la auditoría se repite igual cada vez.	Alguien lee solo la columna exactitud y concluye que el modelo es parejo, sin ver que detecta .26 de los malos con vivienda propia. Además son 200 casos y una sola partición: ninguna cifra trae intervalo de confianza.
Hook pre-commit	Commits con pruebas rotas y con TODO olvidados en archivos que ninguna prueba importa.	git commit --no-verify lo salta. Vive en .git/hooks y no viaja con el repositorio (Chacon y Straub, 2014): quien clone debe correr instalar_hook.py. Y una regla sin prueba no se vigila: verificar_tono marca «alto» con 12 %, pero un texto que diga «considerable» pasa.
Extensión	Comandos o agentes mal formados (manifiesto sin name, comando sin description) y auditorías improvisadas de cada persona.	verificar_extension.py solo revisa la estructura: un comando con description pero con una instrucción equivocada se reporta como válido. El agente es un modelo de lenguaje y puede errar, y el hook no revisa la carpeta extension/.

Nota. Cifras de `python tareas.py vivienda` sobre 200 casos de prueba.

Las tres piezas reducen errores repetibles, pero ninguna sustituye el criterio: el hook solo vigila reglas ya escritas y la extensión no comprueba la calidad de sus instrucciones.

Referencias

Chacon, S., y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book

Hofmann, H. (1994). *Statlog (German Credit Data)* [Conjunto de datos]. UCI Machine Learning Repository. https://doi.org/10.24432/C5NC77